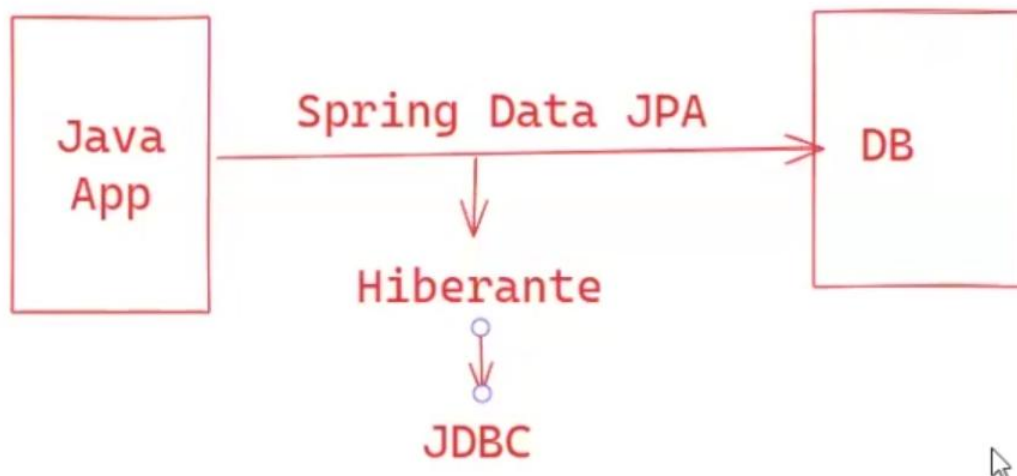


Spring Data JPA

Spring Data JPA

- ❖ JPA : Java Persistence API
- ❖ JPA is used to communicate with Databases using Java applications.
- ❖ We Have several options to communicate with database using java applications
 - 1) JDBC (boiler plate code)
 - 2) Spring JDBC (we have to write queries)
 - 3) Hibernate (ORM -> Obj relational mapping)
 - 4) Spring ORM (internally uses hibernate)
 - 5) Spring Data JPA (latest)



1) What is Data Source

- ❖ DataSource represents set of connections with database (it will create connection pool).
- ❖ Note: We need to configure datasource properties in "application.properties" file.

2) What is Entity

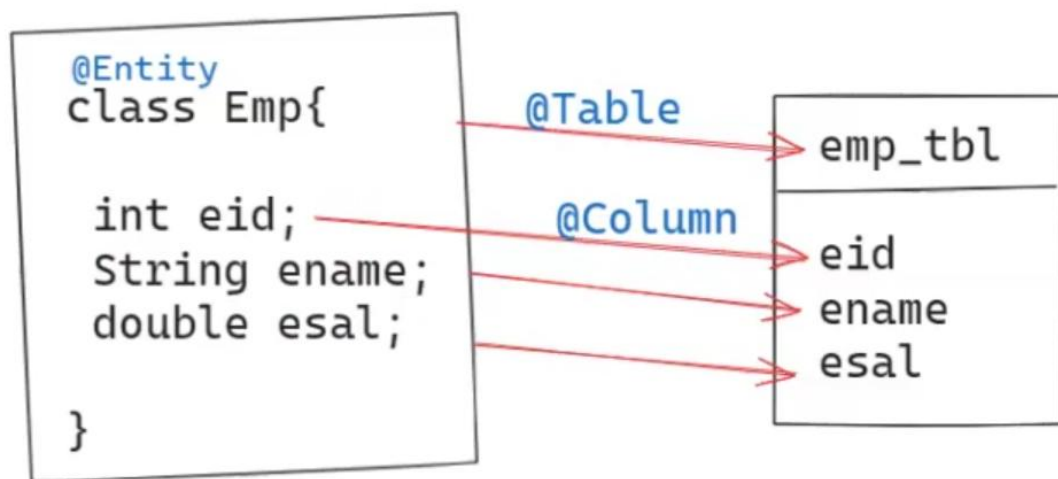
- ❖ Entity is a java class which is mapped with database table. We will use below annotations in entity class.

@Entity : Represent class as entity cls (mandatory)

@Table : map class name with tbl name (optional)

@Id : Represents the variable mapped with PK column (mandatory)

@Column : map cls variable name with tbl column name (optional).



3) What is Repository

❖ Repository is a interface provided by Data JPA to perform CRUD operations.

Note: In data jpa mainley we have 2 repository interfaces

a) CrudRepository : We can perform CRUD Operations

b) JpaRepository : CRUD Ops + Sorting + Pagination + QBE (Query by example)

Developing First Data JPA Application

1) Create Spring Boot application with below dependencies

a) starter-data-jpa

b) mysql-driver

2) Configure datasource properties in application.properties file

```
spring.datasource.url=
```

```
spring.datasource.username=
```

```
spring.datasource.password=
```

```
spring.jpa.show-sql=true
```

```
spring.jpa.properties.hibernate.format_sql=true
```

```
spring.jpa.hibernate.ddl-auto=update
```

3) Create Entity class

```

@Entity
public class Employee {
    @Id
    private int empId;
    private String empName;
    private Double empSalary;
    // setters & getters
}

```

4) Create Our Repository interface by extending Jpa Repository interface.

```

public interface EmpRepository extends CrudRepository<Employee, Integer> {

}

```

5) Test application by calling EmpRepository methods using start class.

Assignment

1) Database Setup

- MySQL DB Server
- MySQL workbench (Client s/w)

Ref : <https://www.youtube.com/watch?v=EsAIXPlsyQg>

2) Developing First Data JPA app to insert emp record into mysql database table.

Ref : <https://www.youtube.com/watch?v=ZGKHCJsp4hg>

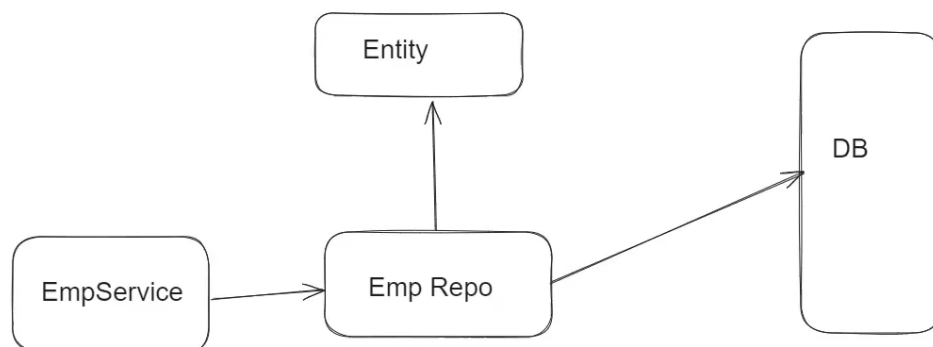
3) Work with all Crud repository methods

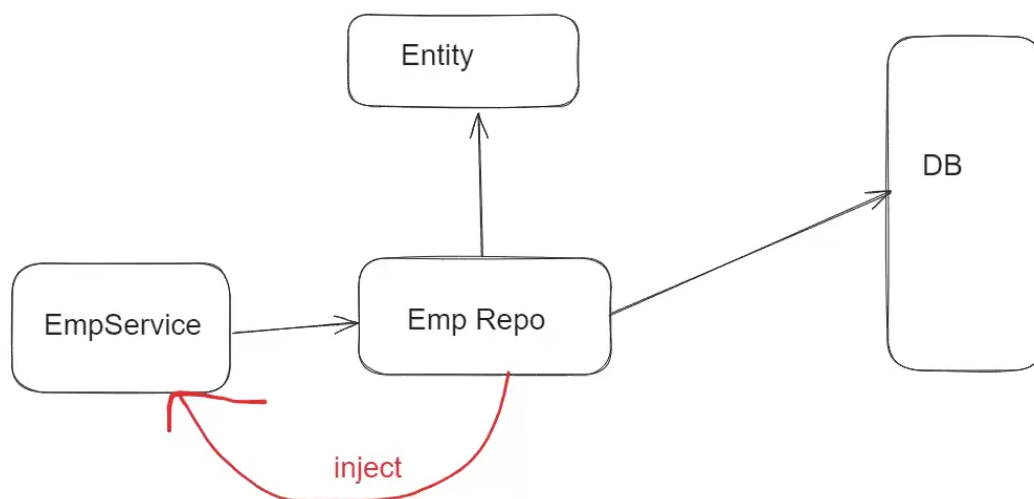
```
Employee emp3 = new Employee();  
emp3.setEmpId(3);  
emp3.setEmpName("Smith");  
emp3.setEmpSalary(5000.00);  
emp3.setEmpGender("Male");  
emp3.setEmpCountry("Canada");
```

```
Employee emp4 = new Employee();  
emp4.setEmpId(4);  
emp4.setEmpName("Robert");  
emp4.setEmpSalary(7000.00);  
emp4.setEmpGender("Male");  
emp4.setEmpCountry("Japan");
```

```
List<Employee> empList = Arrays.asList(emp3, emp4);
```

```
repo.saveAll(empList);|
```





❖ Using Data JPA we can perform operations with Database table in 3 ways

- 1) Predefined Repository methods
- 2) findByXXX methods
- 3) Custom Queries

findByXXX methods

- ❖ findBy methods are used to perform only select operations on the table (We can't use for insert, update and delete).
- ❖ findBy methods are used to execute queries with 'where' clause

Ex : select * from employee where emp_country='India';
 select * from employee where emp_gender='Male';
 select * from employee where emp_salary >= 5000

```
select * from employee where emp_gender='Fe-Male' and emp_country='India'
```

- ❖ When we are using findBy methods, method name will play very important role. Based on method name data jpa will construct query for execution.

Note: If we don't write method name properly then jpa will not accept it.

Note: findBy methods we will write inside Repository interface as abstract methods.

```
package com.ashokit.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class Employee {
    @Id
    private int empId;

    private String empName;

    private Double empSalary;

    public int getEmpId() {
        return empId;
    }

    public void setEmpId(int empId) {
        this.empId = empId;
    }

    public String getEmpName() {
        return empName;
    }

    public void setEmpName(String empName) {
        this.empName = empName;
    }

    public Double getEmpSalary() {
```



```

        return empSalary;
    }

    public void setEmpSalary(Double empSalary) {
        this.empSalary = empSalary;
    }

    @Override
    public String toString() {
        return "Employee{" +
            "empId=" + empId +
            ", empName='" + empName + '\'' +
            ", empSalary=" + empSalary +
            '}';
    }
}

package com.ashokit;

import com.ashokit.service.EmployeeService;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ConfigurableApplicationContext;

@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        ConfigurableApplicationContext context =
SpringApplication.run(Application.class, args);

        EmployeeService service = context.getBean(EmployeeService.class);
//        service.saveEmployee();
//        service.getAllEmployee();

        service.getFindAllByIdEmployee();

        service.findByEmpName("Deep");

        service.findByEmpSalary(77000.0);

        service.findByEmpNameAndEmpSalary("Aman", 77000.0);

        service.findByEmpSalaryGreaterThan(13000.0);
    }
}

```

```
}
```

```
package com.ashokit.repository;

import com.ashokit.entity.Employee;
import org.springframework.data.domain.Pageable;
import org.springframework.data.repository.CrudRepository;

import java.util.List;

public interface EmployeeRepository extends
    CrudRepository<Employee,Integer> {

    // select * from employee where emp_name='Aman';
    public List<Employee> findByEmpName(String name);    // abstract
method - no body

    // select * from employee where emp_salary=77000;
    public List<Employee> findByEmpSalary(Double salary);

    // select * from employee where emp_name='Aman' and emp_salary=77000;
    public List<Employee> findByEmpNameAndEmpSalary(String name,Double
salary);

    // select * from employee where emp_salary > 13000;
    public List<Employee> findByEmpSalaryGreaterThan(Double salary);

}
```

```
package com.ashokit.service;

import com.ashokit.entity.Employee;
import com.ashokit.repository.EmployeeRepository;
import org.springframework.stereotype.Service;

import java.util.Arrays;
import java.util.Iterator;
import java.util.List;

@Service
```

```

public class EmployeeService {

    private EmployeeRepository employeeRepository;

    public EmployeeService(EmployeeRepository employeeRepository){
        System.out.println("EmployeeService() param constructor is
called...");
        this.employeeRepository=employeeRepository;
    }

    public void saveEmployee(){
        Employee employee=new Employee();
        employee.setEmpId(102);
        employee.setEmpName("Aman");
        employee.setEmpSalary(77000.0);

        employeeRepository.save(employee);
    }

    public void getAllEmployee(){
        Iterable<Employee> emps = employeeRepository.findAll();
        emps.forEach(System.out::println); // method reference

        // lambda expression
        emps.forEach(e->{
            System.out.println(e);
        });

        // for each loop
        for (Employee employee:emps){
            System.out.println(employee);
        }

        // iterator
        Iterator<Employee> iterator = emps.iterator();
        while (iterator.hasNext()){
            Employee emp = iterator.next();
            System.out.println("Emp : "+emp);
        }
    }

    public void getFindAllByIdEmployee(){
        Iterable<Employee> allById =
employeeRepository.findAllById(Arrays.asList(1, 2, 3));

        System.out.println("Find All By Id Employee");
        for (Employee employee:allById){

```

```

        System.out.println(employee);
    }

}

public void findByEmpName(String name){
    List<Employee> byEmpName = employeeRepository.findByEmpName(name);
    System.out.println("--- findByEmpName -----");
    System.out.println(byEmpName);
}

public void findByEmpSalary(Double empSalary){
    List<Employee> listOfEmp =
employeeRepository.findByEmpSalary(empSalary);
    System.out.println("--- findByEmpSalary -----");
    System.out.println(listOfEmp);
}

public void findByEmpNameAndEmpSalary(String name, Double salary){
    List<Employee> byEmpNameAndEmpSalary =
employeeRepository.findByEmpNameAndEmpSalary(name, salary);
    System.out.println("--- findByEmpNameAndEmpSalary -----");
    System.out.println(byEmpNameAndEmpSalary);
}

public void findByEmpSalaryGreaterThan(Double salary){
    List<Employee> byEmpSalaryGreaterThan =
employeeRepository.findByEmpSalaryGreaterThan(salary);
    System.out.println("--- findByEmpSalaryGreaterThan -----");
    System.out.println(byEmpSalaryGreaterThan);
}
}

```

Custom Queries

- ❖ To Execute our own queries using data jpa then we will use this Custom Queries concept.
- ❖ To work with custom queries, we will use @Query annotation
- ❖ Custom Queries methods we will write in data jpa repository interface.
- ❖ Custom Queries we can write in 2 ways
 - 1) Native SQL (plain sql)
 - 2) HQL

SQL Queries

- ❖ SQL Queries are db dependent queries.

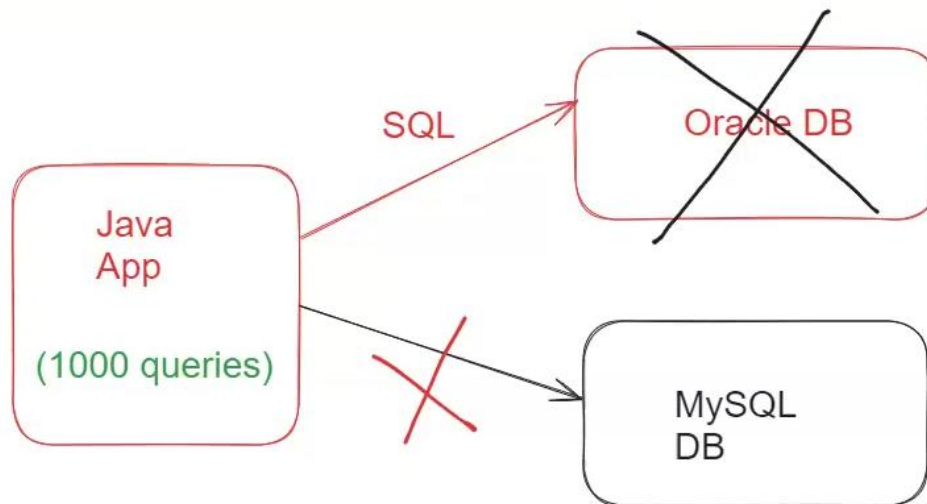
Ex: SQL queries we are writing to communicate with oracle db may not work with mysql db.

- ❖ In SQL query we will use table names and column names directly.

Ex: `select * from emp_data where emp_salary >= 10000;`

Note: If we want to change our application from one DB to another DB then we have to modify SQL queries and we have to re-test entire application.

- ❖ To make our application loosely coupled with database we can use HQL queries.



HQL Queries

- ❖ HQL stands for hibernate query language.
- ❖ HQL queries are database independent.
- ❖ HQL queries will make our app loosely coupled with database.
- ❖ In HQL queries we will use entity class name and entity cls variable names.

Ex : From Employee where empGender='Male';

Note: Database can't understand HQL directly.

- ❖ HQL should be converted to SQL for execution.

- ❖ Dialect classes are used to convert HQL to SQL.
- ❖ Every DB will have its own dialect class

Ex: OracleDialect, MySQLDialect, DB2Dialect....

SQL : select * from emp_tbl

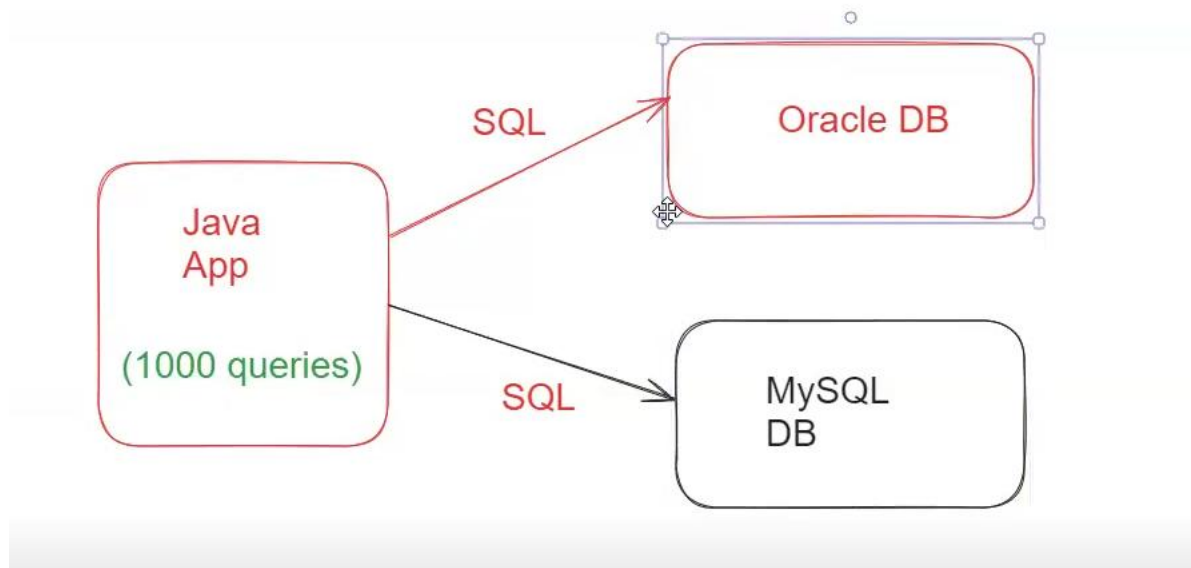
HQL : From Employee

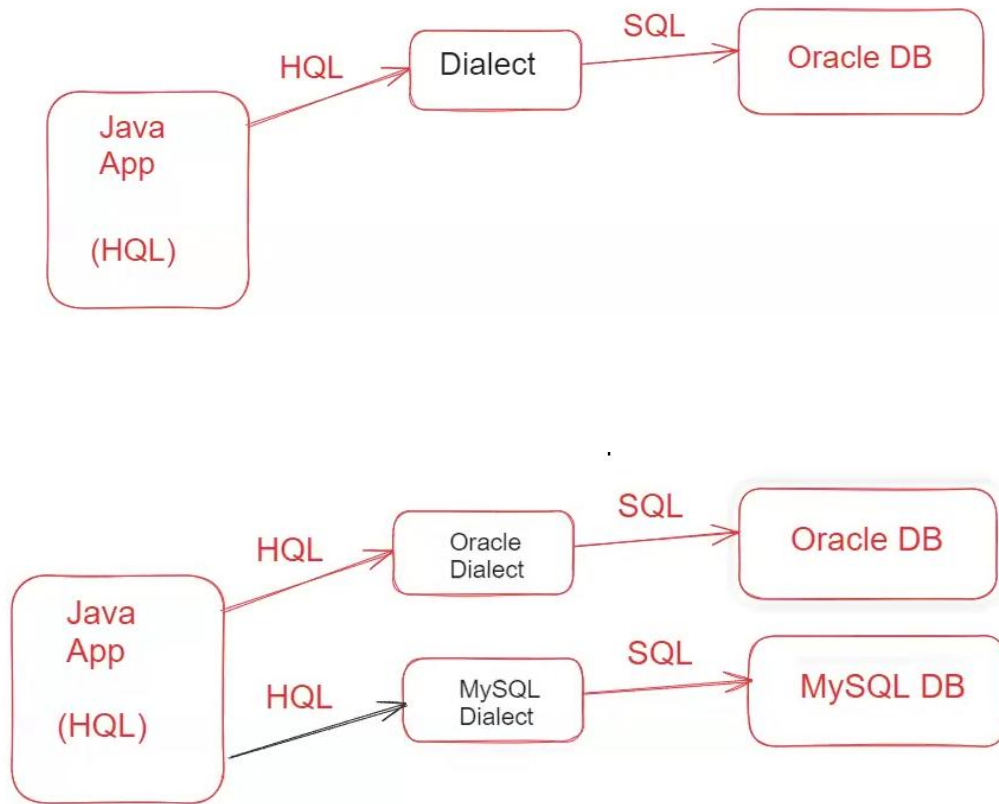
SQL : select * from emp_tbl where emp_id=101;

HQL : From Employee where empId=101

SQL : select emp_id, emp_name from emp_tbl

HQL : select empId, empName from Employee





```
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {
```

```
    @Query("from Employee")
```

```
    public List<Employee> getAllEmps();
```

```
    @Query("from Employee where empGender=:gender")
```

```
    public List<Employee> getEmpsWithGender(String gender);
```

```
    @Query(value = "select * from employee", nativeQuery = true)
```

```
    public List<Employee> getEmpsSQL();
```

```
}
```



```

package com.ashokit.repository;

import com.ashokit.entity.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

import java.util.List;

public interface EmployeeRepository extends
JpaRepository<Employee,Integer> {

    // select * from employee where emp_name='Aman';
    public List<Employee> findByEmpName(String name);    // abstract
method - no body

    // select * from employee where emp_salary=77000;
    public List<Employee> findByEmpSalary(Double salary);

    // select * from employee where emp_name='Aman' and emp_salary=77000;
    public List<Employee> findByEmpNameAndEmpSalary(String name,Double
salary);

    // select * from employee where emp_salary > 13000;
    public List<Employee> findByEmpSalaryGreaterThan(Double salary);

//    public List<Employee> findByEmpSalaryGrea(Double salary);

    // ----- custom query example -----
    // HQL
    @Query("from Employee")
    public List<Employee> getAllEmpsHql();

    // SQL
    @Query(value = "select * from employee",nativeQuery = true)
    public List<Employee> getAllEmpsSql();

    // HQL
    @Query("from Employee where empName=:name")
    public List<Employee> getEmpsWithNameHql(String name);
}

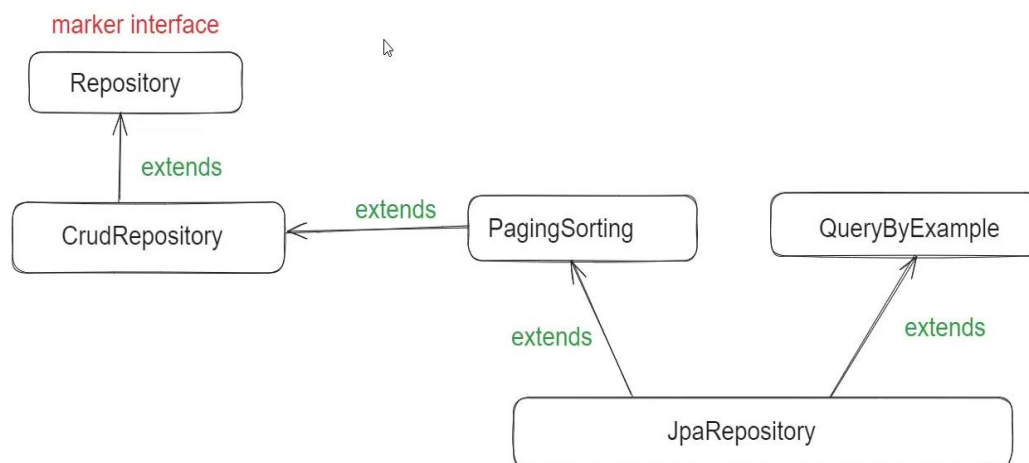
```

Assignment

1) Implement insert + update + delete using Custom HQL queries.

Q) Which is better SQL or HQL ?

- ❖ Performance wise SQL is better
- ❖ Flexibility wise HQL is better



JpaRepository

- ❖ This is a predefined data jpa interface which is used to perform DB operations.
- ❖ Using Jpa repository we can perform below operations
 - a) Crud operations
 - b) Sorting
 - c) Pagination
 - d) Query By Example (dynamic query creation)

JpaRepository = CrudRepository + Sorting + Pagination + QBE

```
package com.ashokit.service;

import com.ashokit.entity.Employee;
import com.ashokit.repository.EmployeeRepository;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class EmployeeService {

    private EmployeeRepository employeeRepository;

    public EmployeeService(EmployeeRepository employeeRepository){
        this.employeeRepository=employeeRepository;
    }

    public void findAllEmp(){
        // List<Employee> all = empRepo.findAll();
        List<Employee> empNameList =
employeeRepository.findAll(Sort.by("empName").descending());
        empNameList.forEach(System.out::println);
    }

}
```

What is Pagination ?

❖ The process dividing records into multiple pages.

Ex: If we have 1 lakh records in table it is not recommended to display one lakh records in single page.

❖ In realtime we need to implement pagination in our application.

gmail ---> will display 50 records in inbox

flipkart --> will display 24 records in one page

`page_size = 3`

`total_pages = ?`



`total_records/page_size = total_pages`



page_size = 10

total_pages = ?

10
records

$\text{total_records} / \text{page_size} = \text{total_pages}$

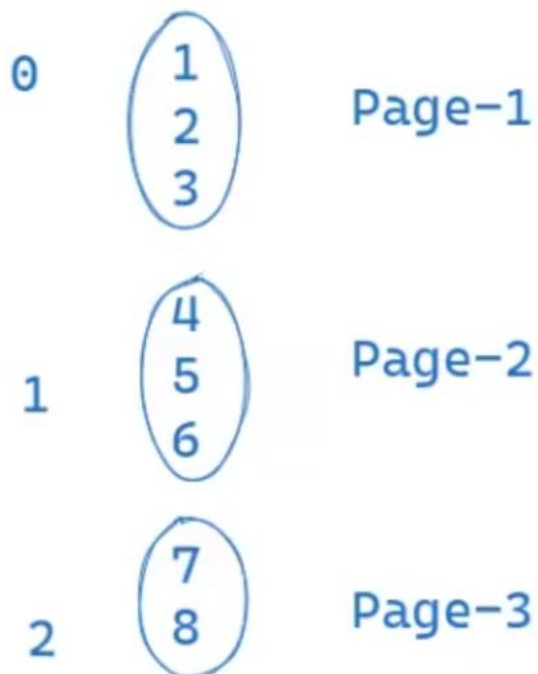
$10/10 = 1 \text{ page}$

$10/3 = 3 + 1 \text{ pages}$

$10/5 = 2 \text{ pages}$

DB Count

User Count



```

package com.ashokit.service;

import com.ashokit.entity.Employee;
import com.ashokit.repository.EmployeeRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class EmployeeService {

    private EmployeeRepository employeeRepository;

    private static final int pageSize=3;

    public EmployeeService(EmployeeRepository employeeRepository){
        this.employeeRepository=employeeRepository;
    }

    public void findAllEmp(){
        // List<Employee> all = empRepo.findAll();
        List<Employee> empNameList =
employeeRepository.findAll(Sort.by("empId").ascending());
        empNameList.forEach(System.out::println);
    }

    public void getEmpPage(int pageNum){
        System.out.println(" ----- Pagination ----- ");
        Pageable page= PageRequest.of(pageNum - 1, pageSize);

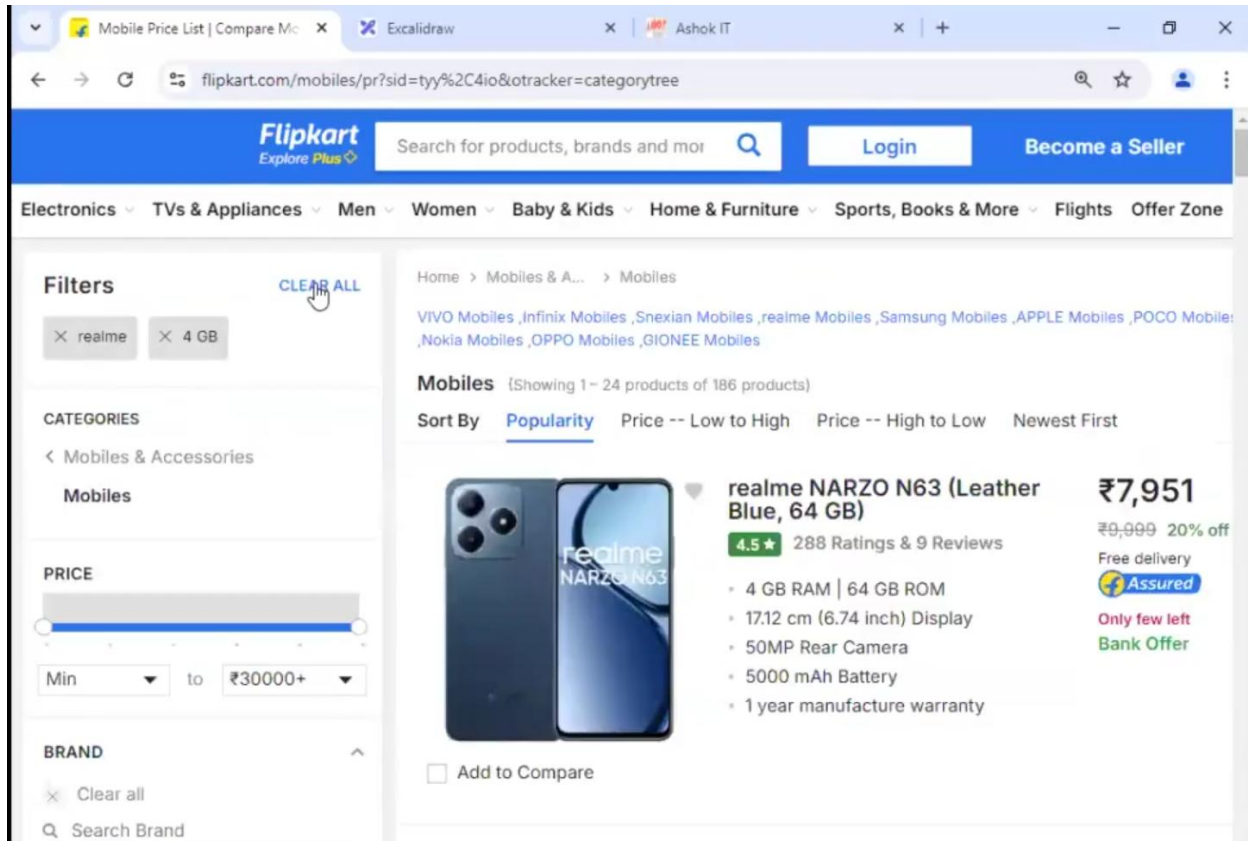
        Page<Employee> emps = employeeRepository.findAll(page);

        emps.get().forEach(System.out::println);
    }
}

```

Query By Example

- ❖ It is used to construct dynamic query based on filter conditions.
- ❖ Based on entity obj data query will be created.



```
package com.ashokit;

import com.ashokit.entity.Employee;
import com.ashokit.service.EmployeeService;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```

import org.springframework.context.ConfigurableApplicationContext;

@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        ConfigurableApplicationContext context =
        SpringApplication.run(Application.class, args);

        EmployeeService employeeServiceBean =
context.getBean(EmployeeService.class);
        // sort
        employeeServiceBean.findAllEmp();

        // pagination
        employeeServiceBean.getEmpPage(3);

        // Query By Example
        Employee employee=new Employee();
        employee.setEmpName("Deep");
        employee.setEmpSalary(13000.0);

        employeeServiceBean.getEmpsWithQBE(employee);

    }
}

```

```

package com.ashokit.service;

import com.ashokit.entity.Employee;
import com.ashokit.repository.EmployeeRepository;
import org.springframework.data.domain.*;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class EmployeeService {

    private EmployeeRepository employeeRepository;

    private static final int pageSize=3;

    public EmployeeService(EmployeeRepository employeeRepository){

```



```

        this.employeeRepository=employeeRepository;
    }

    // sort
    public void findAllEmp(){
        System.out.println(" ----- Sorting ----- ");
        // List<Employee> all = empRepo.findAll();
        List<Employee> empNameList =
employeeRepository.findAll(Sort.by("empId").ascending());
        empNameList.forEach(System.out::println);
    }

    // pagination
    public void getEmpPage(int pageNum){
        System.out.println(" ----- Pagination ----- ");
        Pageable page= PageRequest.of(pageNum - 1, pageSize);

        Page<Employee> emps = employeeRepository.findAll(page);

        emps.get().forEach(System.out::println);
    }

    // Filter - Query By Example
    public void getEmpsWithQBE(Employee emp){
        System.out.println(" ----- Query By Example ----- ");
        Example<Employee> employeeExample = Example.of(emp);

        List<Employee> employees =
employeeRepository.findAll(employeeExample);

        employees.forEach(System.out::println);
    }
}

```

Assignment

Requirement : Develop data jpa application to retrieve employees based on below conditions

Condition : Male Employees belongs to India having salary >=5000

Database Relationships (Association Mapping)

❖ We can see below relationship with db tables

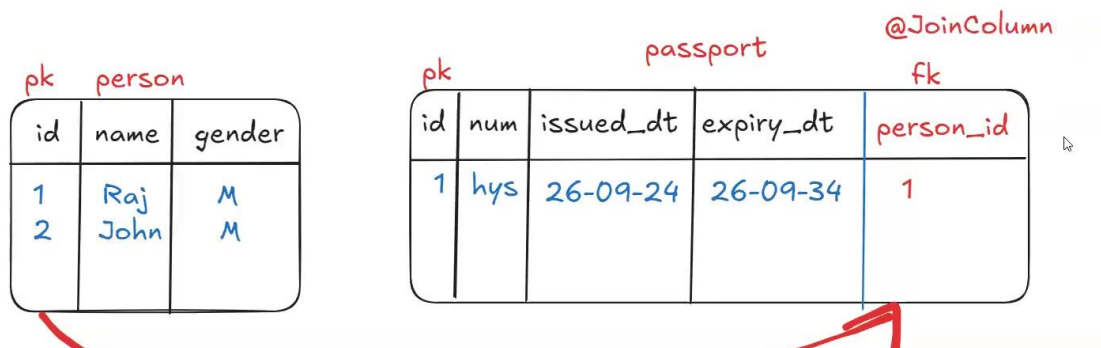
1) One To One

Ex: Person & Passport

Note: One record in parent table will have relationship with one record in child table.

Ex: One person will have one passport.

❖ To represent one to one relationship among entities we will use @OneToOne annotation



2) One To Many

Ex : Employee with addresses

Ex : Author with Books

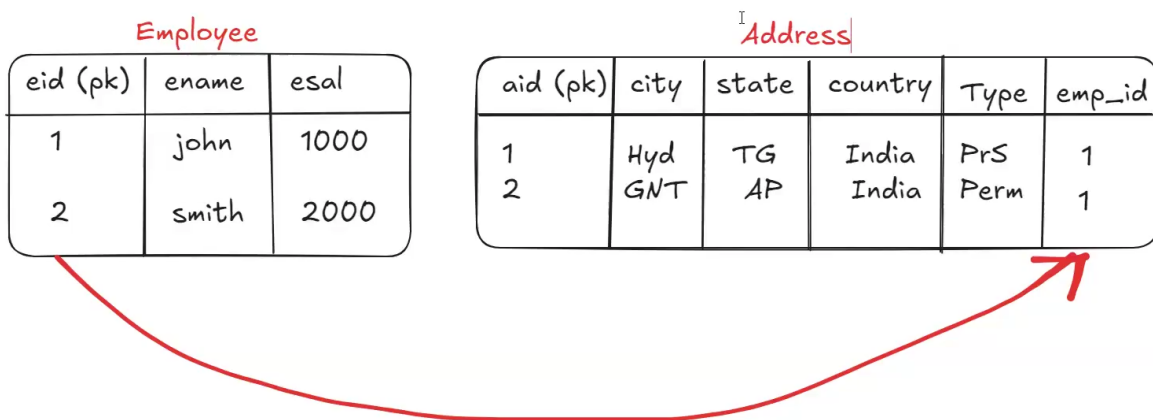
Ex : Trainer with Courses

Ex : Category with Products

Note: One record in parent table will have relationship with multiple records in child table

Ex: One employee will have multiple addresses (present & permanent)

❖ To represent one to many relationship we will use @OneToMany annotation.

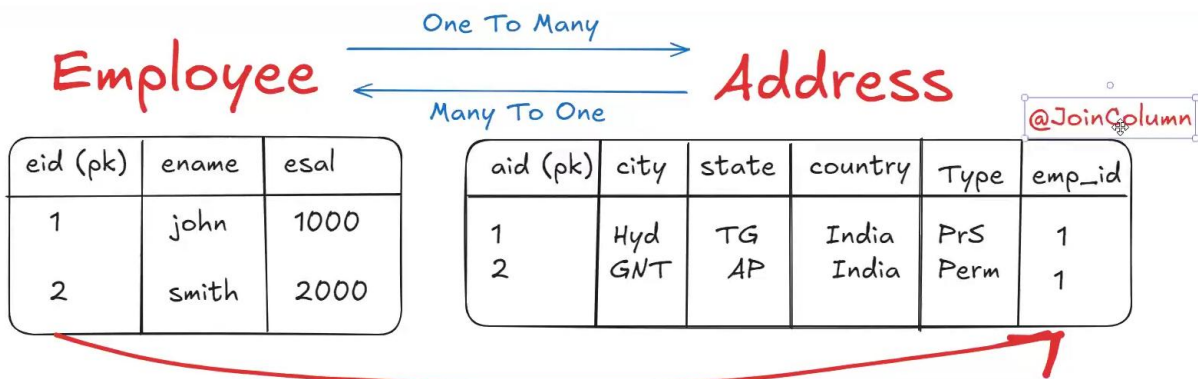


3) Many To One

Ex: Multiple books with single author

Note : Many records in one table will have relationship with single record in another table.

❖ To represent Many To One relationship we will use @ManyToOne annotation.



4) Many To Many

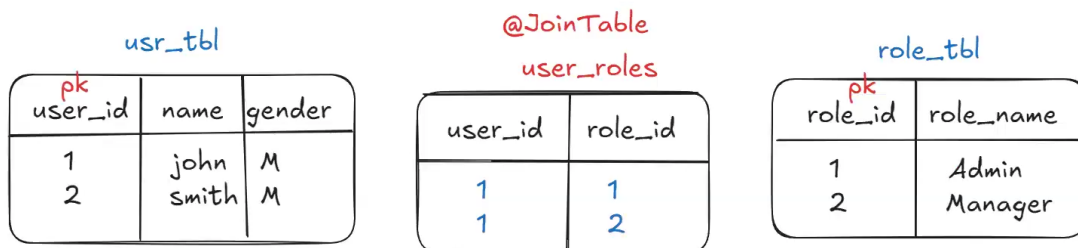
Ex: Users with Roles

Note: Multiple users will have relationship with multiple roles.

user_tbl : users data will be stored

role_tbl : roles data will be stored

user_roles : join table



- ❖ When DB tables having relationships then we have to represent those relationships in Entity classes which is called as Association Mapping.
- ❖ To establish association mapping in entity classes we will use below annotations...

@OneToOne

@OneToMany

@ManyToOne

@ManyToMany

@JoinColumn

@JoinTable

What is Cascade ?

- ❖ Parent table Operations should reflect on child table or not will be represented by CASCADE.

What is Fetch Type ?

- ❖ Fetch Type represents child records should be loaded along with parent record or not.
- ❖ We have below 2 fetch types
 - Lazy (default)
 - Eager
- ❖ Lazy means child records will be retrieved on demand basis.
- ❖ Eager means child records will be retrieved along with parent record in single query.

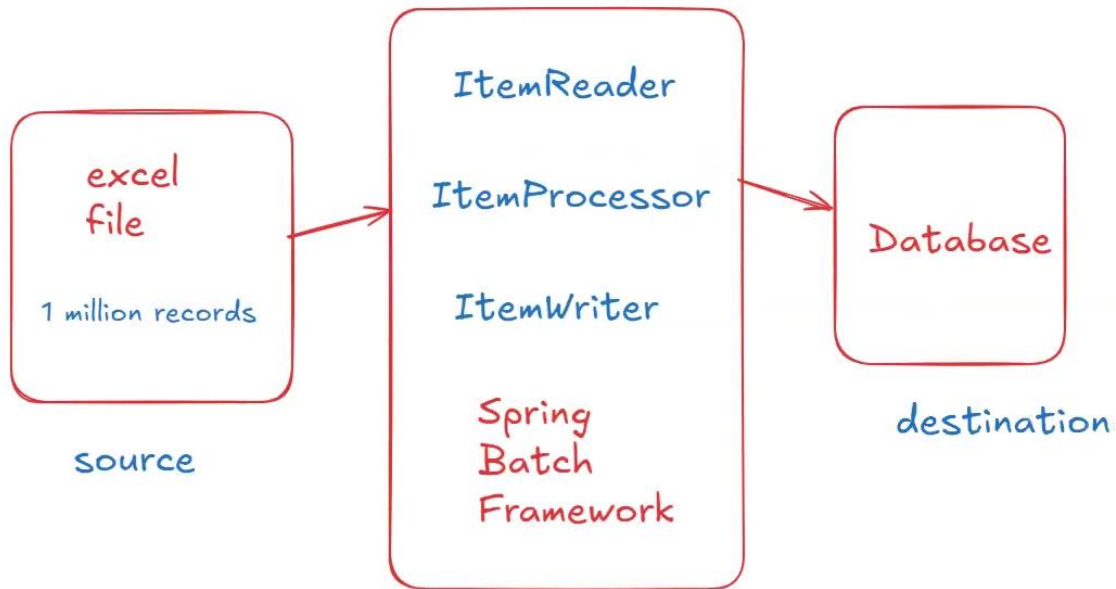
One to one : https://github.com/ashokitschool/springboot_jpa_one_to_one_app.git

one to many : https://github.com/ashokitschool/springboot_jpa_one_to_many_app.git

many to many : https://github.com/ashokitschool/springboot_jpa_many_to_many_app.git

Spring Batch

- ❖ Spring Batch is a powerful framework for processing large volumes of data in a batch-oriented way.
- ❖ It's part of the Spring Framework and provides reusable functions essential for processing data, such as logging, transaction management, job processing statistics, and more.
- ❖ Here are some key components and concepts of Spring Batch:
 - ItemReader
 - ItemProcessor
 - ItemWriter
 - Step
 - Job
 - JobRepository
 - JobLauncher



ItemReader : It is used to read data from the source

ItemProcessor : It is used to process the data

ItemWriter : It is used to write the data to destination

Step : It represents sequence of our batch execution (reader + processor + writer)

Job : It is used to represents set of steps for batch processing

JobRepository : To maintains job execution history and steps info

JobLauncher: It is used to start the job exeuction.

Batch Processing Use cases

- bulk msg
- bulk email
- sending salaries emps

- sending bank stmt to acc holders
- sending post paid bill reports
- Sending payment reminder notifications

Requirement

1) Read the data from csv file and write into database table using spring batch framework.

Note: Skip the records which are having price as 0.

Ex:

ISBN001, Java, 1000.00

ISBN002, Python, 2000.00

ISBN003, DevOps, 3000.00

ISBN004, AWS, 4000.00

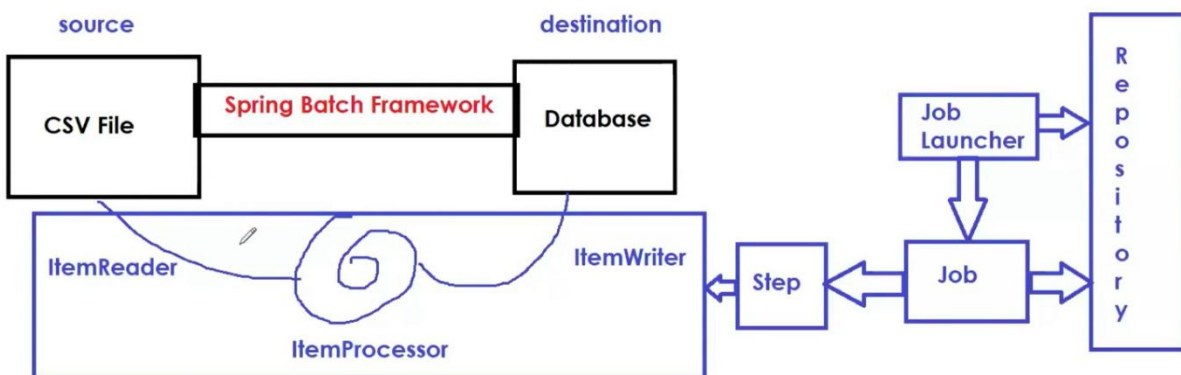
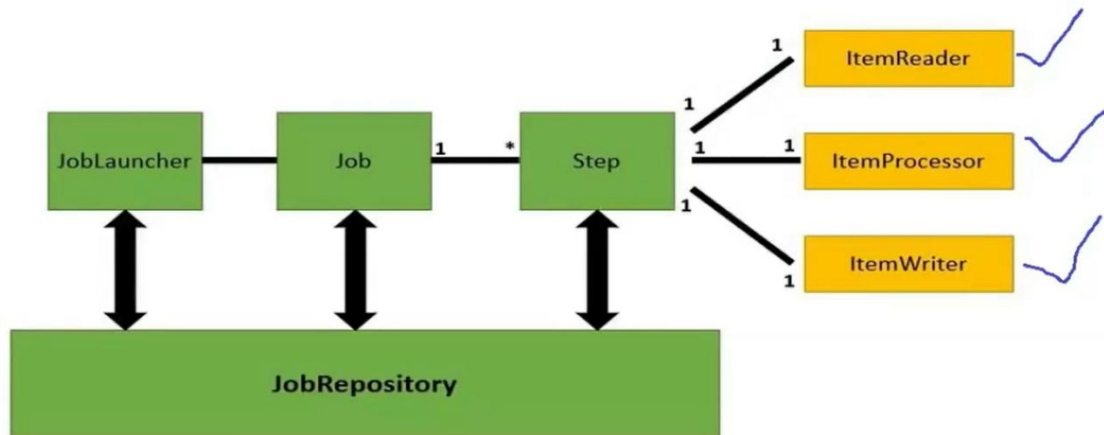
ISBN005, Docker, 0.00

ISBN006, AI, 5000.00

Spring Batch Tutorial : <https://youtu.be/2sdY-fKaR2o>

Spring Doc: <https://spring.io/guides/gs/batch-processing>

SPRING BATCH ARCHITECTURE



What is Tx management

- ❖ Single Unit amount of work
- ❖ use case :
 - ◆ Transfer amount from one account to another account
 - a) debit money from sender account
 - b) credit money to reciever account
- ❖ If above two operations got completed then we will consider that as successfull transaction.

Note: When tx is successfull then we have to commit that transaction.

```
tx.commit ( )
```

- ❖ If any one operation got failed then we will consider that as failure transaction.

Note: When tx is failure then we have to rollback that transaction.

```
tx.rollback ( )
```

```
@Transactional(rollbackFor = Exception.class)
```

```
public void saveEmp() {
```

```
    Emp e = new Emp();
```

```
    e.setName("chary");
```

```
    e.setSalary(3000.00);
```

```
Emp savedEmp = empRepo.save(e);  
System.out.println(savedEmp);  
  
int i = 10 / 0;  
  
Address addr = new Address();  
addr.setCity("Pune");  
addr.setState("MH");  
addr.setCountry("India");  
addr.setEid(savedEmp.getEid());  
  
addrRepo.save(addr); // saving addr  
  
}
```

Transaction Propagation

- ❖ Tx Propagation defines how tx is related between calling and called methods.
- ❖ In below example, m1() is calling m2 ()

m1 () => calling method

m2 () => called method

```
@Transactional  
public void m1() {  
    m2();  
}
```

```
@Transactional  
public void m2() {  
  
}
```

- ❖ Below are the main propagation types:
- ❖ REQUIRED (default): If a transaction exists in calling method then use it for called method also. If not available then create new tx for called method.
- ❖ REQUIRES_NEW : Always create a new transaction. Suspends the current transaction if one exists.
- ❖ SUPPORTS : If a transaction exists, use it; if not, execute non-transactionally.
- ❖ NOT_SUPPORTED: Always executes non-transactionally, suspending any existing transaction.
- ❖ MANDATORY : Requires an existing transaction. Throws an exception if none exists.
- ❖ NEVER : Execute non-transactionally, throw an exception if a transaction exists.

- ❖ NESTED : Execute within a nested transaction if a current transaction exists, behave like REQUIRED otherwise.

Transaction Isolation

- ❖ Tx isolation defines how and when changes made by one tx visible to another tx.
- ❖ The main levels of isolation in JPA are
- ❖ READ_COMMITTED : Prevent dirty reads, tx can read only committed data.
- ❖ READ_UNCOMMITTED : Allow dirty reads, tx can read un committed changes also from other tx.
- ❖ REPEATABLE_READ : Ensures that if a transaction reads a row, subsequent reads will return the same data, preventing non-repeatable reads.
- ❖ SERIALIZABLE : The strictest isolation level, ensuring complete isolation from other transactions

Assignment – Solution

Spring Data JPA

1) create, update, delete records using custom query

- @^IModifying
- @Transactional

2) Retrieve employees based on country and gender using OR clause

- Specification